Phase 0 — Project Scaffold & Local Model Load

Goal of this phase: stand up a clean, professional repo and prove the model loads and generates on a local machine (CPU/Apple Silicon) at zero cost — the foundation everything else builds on.

---

1. What I built

- A structured repo separating concerns: `engine/` (inference internals), `server/` (HTTP layer), `benchmark/`, `deploy/`, `dashboard/`, `tests/`, `scripts/`.
- A reusable model loader (`engine/model.py`) that downloads and caches `Qwen/Qwen2.5-0.5B-Instruct`, auto-detects the best device (CUDA → MPS → CPU), and exposes a simple `generate()` helper.
- A smoke-test CLI (`scripts/smoke_generate.py`) and a pytest suite proving the model loads and produces non-empty output.
- Dependency pinning (`requirements.txt`), `.gitignore` that blocks model weights and venvs, and a starter README.

Verification: `pytest -q` → 2 passed. `smoke_generate.py` produces a correct one-sentence answer in ~3s on Apple Silicon (MPS).

---

2. Why — design decisions (the part interviewers probe)

Decision	Why
`engine/` separate from `server/`	The inference internals (KV cache, scheduler) are the intellectual core and must be unit-testable without HTTP. The server is a thin transport over the engine. This separation is what lets Phase 3 swap engine implementations behind one API.
Loader in `engine/`, not inline in the script	Phases 1–5 share one loading path → consistent dtype/device → fair, apples-to-apples benchmarks.
`@lru_cache` on the loader	Model weights load once per process, not once per request. A 0.5B model is ~1 GB; reloading per request would dominate latency.
Auto device detection	Same code runs free on my Mac (MPS/CPU) for dev and on a cloud GPU (CUDA) for real benchmarks — no code changes between environments.
fp32 on CPU, fp16 on GPU/MPS	CPUs have no fast fp16 path, so fp32 is both correct and faster there; fp16 halves memory and speeds up matmuls on GPU.
Weights never committed (`.gitignore`)	Repos with gigabytes of weights are an instant red flag and break clones; we download from HF at runtime.

Decision	Why
Model = Qwen2.5-0.5B-Instruct	Ungated (no HF license approval friction) and tiny, so CPU iteration is fast. The architecture lessons (KV cache, batching) are identical at any size.
Greedy decoding (`do_sample=False`) in tests	Deterministic output → tests can assert correctness reproducibly.

A real bug I fixed

The model loaded **twice** per process. `@lru_cache` keys on the *exact* arguments, so `load_model()` and `load_model("Qwen/...")` were different keys despite resolving identically. Fix: resolve arguments to concrete values **first**, then cache on those. Lesson: even caching has correctness footguns — the cache key must reflect the resolved inputs, not the call syntax.

---

3. Interview Q&A

Q: Walk me through what happens when you call `model.generate()`. A: The prompt is tokenized into input IDs and run through the model to produce logits over the vocabulary for the next token. We pick a token (greedy = argmax), append it to the sequence, and feed the whole sequence back in to predict the next token — repeating autoregressively until EOS or `max_new_tokens`. Each step is one forward pass. (Phase 2 shows why naive re-feeding is wasteful and how the KV cache fixes it.)

Q: What's the difference between a base model and an `-Instruct` model? A: The base model is pretrained on next-token prediction over raw text. The Instruct variant is further fine-tuned (SFT + preference tuning) to follow chat-formatted instructions. That's why I apply the tokenizer's **chat template** — it wraps the prompt in the special role tokens the model was tuned on; skipping it degrades output quality.

Q: Why fp16 on GPU but fp32 on CPU? A: GPUs have dedicated half-precision/tensor-core hardware, so fp16 roughly halves memory and speeds up matmuls with negligible quality loss for inference. CPUs lack fast fp16 kernels, so fp16 there is emulated and slower — fp32 is the right call.

Q: How much memory does this model need? A: Params × bytes/param. 0.5B params × 2 bytes (fp16) ≈ 1 GB just for weights; ×4 (fp32) ≈ 2 GB. On top of that you pay for activations and the **KV cache**, which grows with sequence length and batch size — that's the dominant serving cost and the subject of Phase 2.

Q: Why not just use the OpenAI/Anthropic API? A: The point is to demonstrate the *systems* underneath — KV caching, batching, GPU utilization, quantization, autoscaling. An API call hides all of that. Self-hosting a small open model surfaces every tradeoff a production inference team actually manages.

Q: What does `model.eval()` and `torch.inference_mode()` do? A: `eval()` switches layers like dropout/batchnorm into inference behavior. `inference_mode()` (a stronger `no_grad()`) disables autograd bookkeeping, saving memory and time since we never backprop during serving.

---

4. One-line recall

> *"Phase 0 is the clean foundation: a separated engine/server repo, a cached auto-device model loader, and a passing smoke test proving Qwen2.5-0.5B generates locally — built so every later optimization is measured on identical, fair footing."*